Arquitectura del Sistema Distribuido — UPB-CIENTÍFICA

1. Razonamiento arquitectónico

1.1 Por qué SOA y no microservicios ni monolito

El sistema integra tecnologías heterogéneas por requisito obligatorio: **PHP/SOAP** (autenticación), **Java/RMI** (HPC), **Go/gRPC** (sincronización), más clientes nativos en cinco plataformas distintas. Esta heterogeneidad es exactamente el escenario para el que SOA fue pensado:

- **Interoperabilidad por contrato, no por implementación.** Cada servicio expone una interfaz bien definida (WSDL en SOAP, IDL en RMI, `.proto` en gRPC) que oculta el lenguaje y la plataforma internos. Un cliente Android no necesita saber que el servicio de autenticación está escrito en PHP.
- **No es microservicios** porque no buscamos despliegue independiente por contenedor, versionado agresivo por servicio ni una base de datos por servicio con eventual consistency como filosofía central. Buscamos **servicios empresariales de grano grueso** (Authentication, HPC, Storage, Sync, Sharing, Photos, Streaming, Monitoring) que se reutilizan entre sí y se orquestan mediante contratos estables, típico de SOA clásico sobre infraestructura propia (no cloud-native).
- **No es monolito** porque cada responsabilidad de negocio (auth, cómputo HPC, almacenamiento, sync, multimedia, monitoreo) vive en un proceso y una base de código independientes, con su propia capa de persistencia, desplegable y escalable por separado dentro del Centro de Computación Avanzada.

1.2 Por qué arquitectura por capas dentro de cada servicio

SOA resuelve el problema *entre* servicios (bajo acoplamiento, alta cohesión a nivel de negocio). Pero *dentro* de cada servicio seguimos necesitando separación de responsabilidades para que sea mantenible y testeable. Por eso cada servicio implementa internamente:

Capa	Responsabilidad
Presentación / API	Expone el contrato SOAP/RMI/gRPC/HTTP. Traduce mensajes externos a llamadas internas. No contiene lógica de negocio.
Aplicación	Orquesta casos de uso (ej. "enviar trabajo HPC"), coordina transacciones, valida entrada a nivel de flujo.
Dominio	Reglas de negocio puras: entidades, invariantes (ej. cuotas de almacenamiento, estados válidos de un job). Sin dependencias de infraestructura.
Persistencia / Infraestructura	Acceso a datos, colas, sistema de archivos, llamadas a MPI, etc. Implementa interfaces definidas por el dominio.

Servicios con necesidades especiales agregan capas adicionales, justificadas:

- **HPC Service** agrega una **capa de Scheduling** entre Aplicación y Dominio, porque la planificación de recursos (cola, asignación de nodos, balanceo) es un sub-dominio con su propio ciclo de vida y no debería mezclarse con las reglas de negocio de "qué es un trabajo válido".
- **File Sync Service** agrega una **capa de Detección de Cambios / Reconciliación** (diffing, versionado, resolución de conflictos) antes de la capa de dominio, porque es un problema algorítmico distinto de las reglas de sincronización en sí.
- **Distributed Storage Service** agrega una **capa de Replicación/Metadatos** separada de la persistencia física, porque la replicación y el versionado son responsabilidades transversales al contenido almacenado.

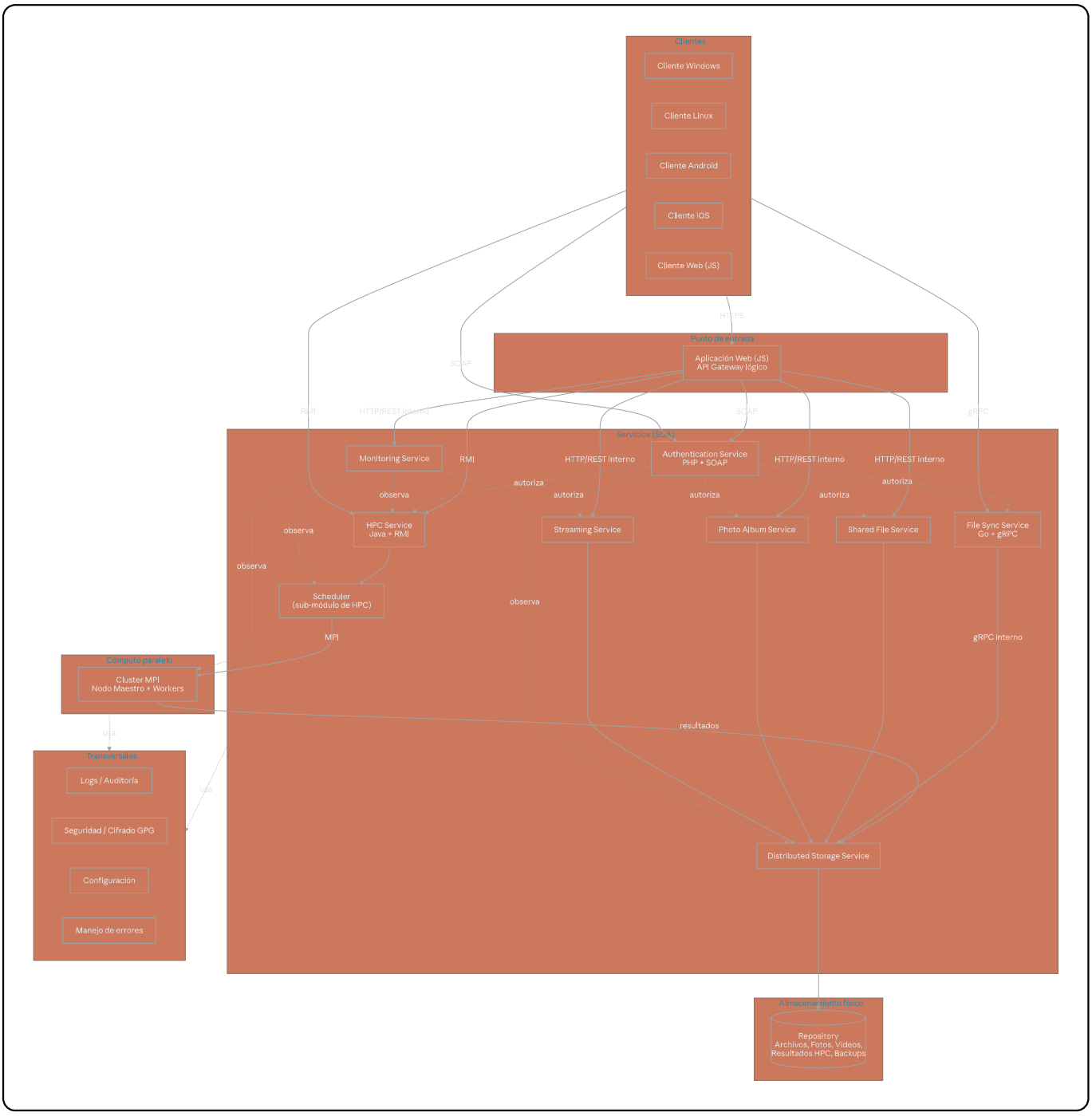
1.3 Cumplimiento de principios

- **Alta cohesión:** cada servicio agrupa únicamente las responsabilidades de su dominio (auth no sabe de streaming, HPC no sabe de fotos).
- **Bajo acoplamiento:** los servicios no comparten base de datos ni código; se comunican solo por contrato (SOAP/RMI/gRPC/HTTP) y a través del **Authentication Service** para autorización.
- **Separación de responsabilidades:** capas internas + servicios externos independientes.
- **Reutilización:** Authentication Service es consumido por todos; Distributed Storage Service es consumido por Sync, Sharing, Photos, Streaming y HPC (para resultados).
- **Escalabilidad:** cada servicio escala horizontalmente de forma independiente (más nodos de Storage, más Workers MPI, más réplicas del Auth Service) sin afectar a los demás.

- **Mantenibilidad:** contratos estables permiten cambiar la implementación interna de un servicio sin romper a los consumidores.

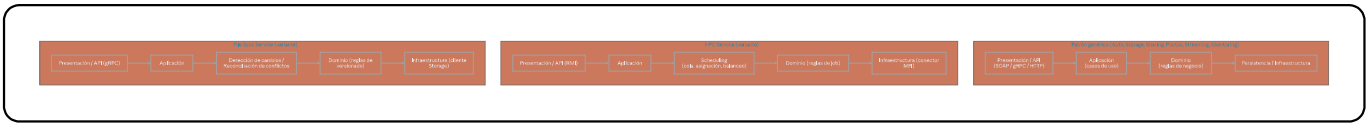
---

2. Diagrama de topología general (SOA)



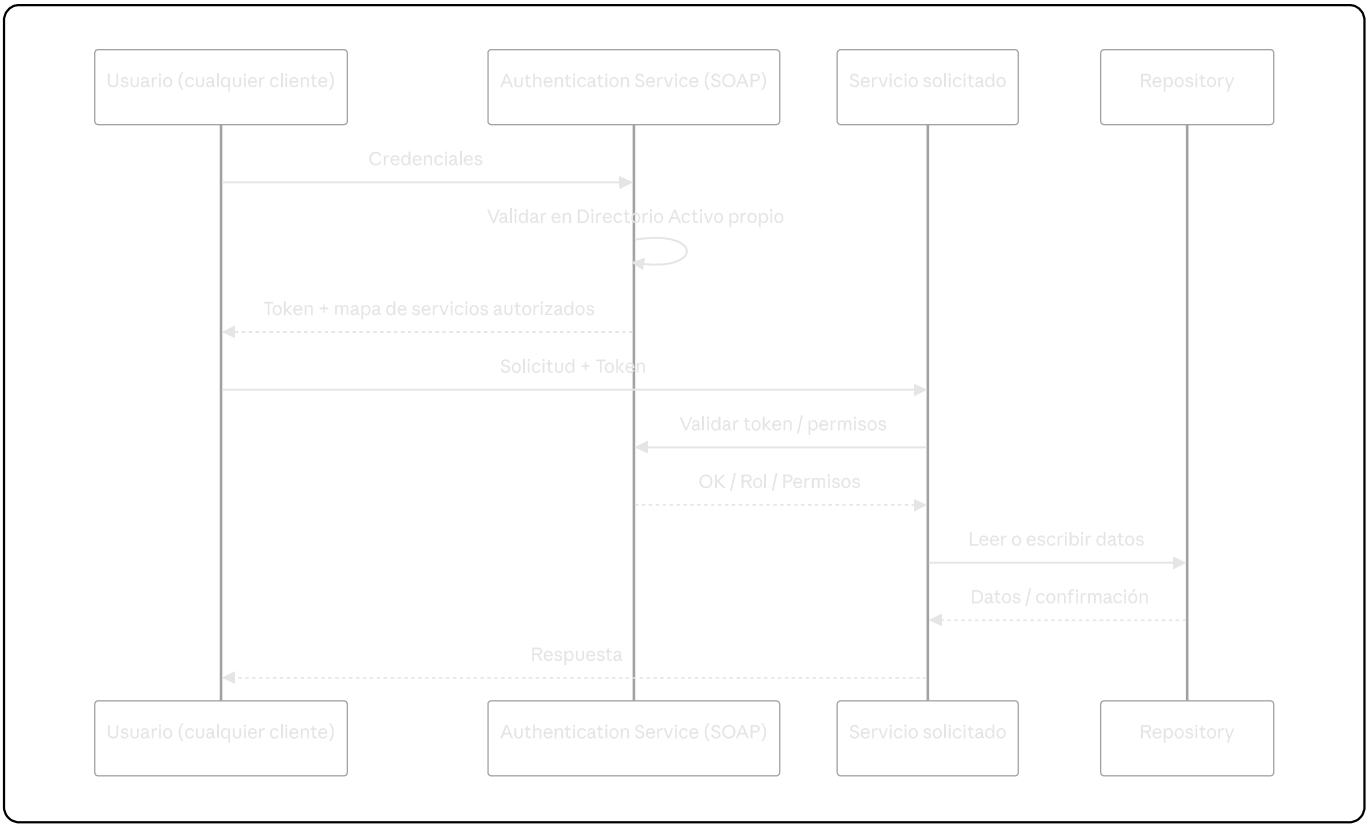
---

3. Arquitectura interna por capas (patrón genérico + variantes)

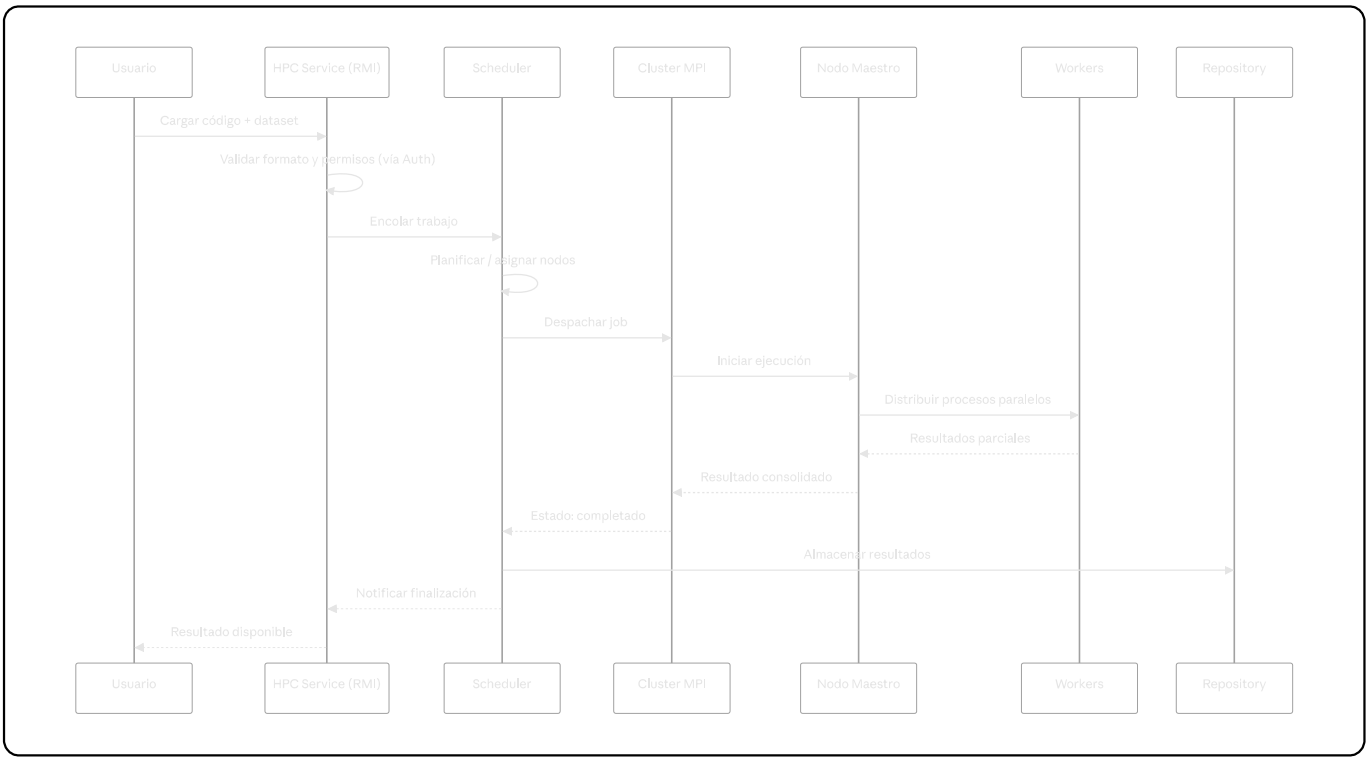


---

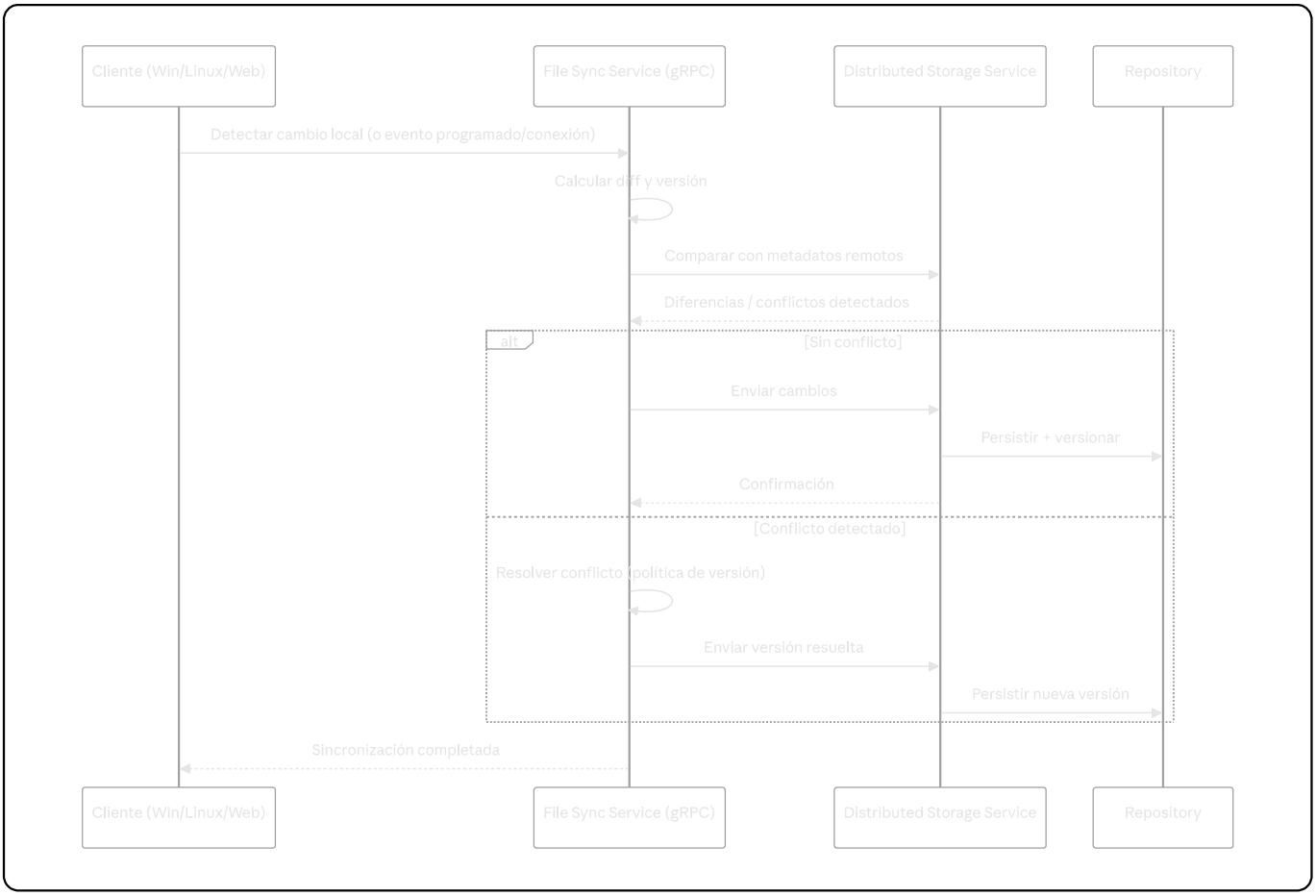
4. Flujo de autenticación y acceso general



5. Flujo de un trabajo HPC (MPI)



6. Flujo de sincronización de archivos



---

7. Aspectos transversales

Componentes reutilizados horizontalmente por todos los servicios, implementados como librerías/módulos compartidos (no como un servicio adicional que rompería el bajo acoplamiento):

- **Logs y Auditoría:** cada servicio escribe en un formato común; el Monitoring Service los consume.
- **Seguridad / Cifrado GPG:** cifrado de archivos sensibles en tránsito y en reposo dentro del Repository.
- **Tokens y Autorización:** emitidos por Authentication Service, validados localmente por cada servicio (evita un cuello de botella de validar contra Auth en cada llamada, si se cachea la validez del token).
- **Configuración centralizada:** parámetros de cada servicio (endpoints, timeouts, cuotas) gestionados de forma consistente.
- **Manejo de errores:** convenciones comunes de códigos de error entre SOAP, RMI y gRPC para que los clientes reaccionen de forma uniforme.

---

8. Análisis de riesgos

8.1 Posibles cuellos de botella

- **Authentication Service:** si todos los servicios validan cada request contra Auth en tiempo real, se vuelve un punto de contención. *Mitigación:* tokens firmados con expiración corta, validación local sin round-trip constante.
- **Scheduler:** al ser el único punto de entrada al clúster MPI, una alta concurrencia de trabajos puede generar cola larga. *Mitigación:* políticas de prioridad y particionamiento de colas por tipo de trabajo.
- **Distributed Storage Service:** es consumido por Sync, Sharing, Photos, Streaming y HPC simultáneamente — el servicio más "reutilizado" es también el más expuesto a saturación de I/O.

8.2 Puntos únicos de fallo (SPOF)

- **Authentication Service** sin réplica: si cae, ningún servicio puede autorizar nada. *Mitigación:* réplica activa-pasiva del directorio.
- **Nodo Maestro MPI:** coordina a los Workers; su caída detiene trabajos en curso. *Mitigación:* checkpointing y nodo maestro secundario.
- **Repository centralizado:** aunque es "distribuido" en almacenamiento, si el metadato/índice central falla, todo el sistema pierde acceso a datos. *Mitigación:* replicación de metadatos, no solo de contenido.

8.3 Riesgos de escalabilidad

- Acoplar Scheduler dentro del HPC Service limita escalarlo de forma totalmente independiente; si crece mucho la demanda de cómputo, convendría separarlo en el futuro.
- El uso de SOAP para autenticación es más pesado (XML, mayor overhead) que gRPC; bajo alta concurrencia de logins puede ser el servicio más lento en responder.
- Streaming de video es intensivo en ancho de banda; sin un mecanismo de cacheo o CDN interna, compite por red con la sincronización de archivos y las transferencias de resultados HPC.

8.4 Mejoras futuras

- Cache distribuida de tokens de sesión para reducir carga sobre Auth.
- Separar el Scheduler como servicio independiente si el volumen de trabajos crece.
- Índice de metadatos replicado (no solo el contenido) en el Repository.
- Circuit breakers entre servicios para degradar con gracia si Storage o Auth responden lento.
- Balanceo de carga a nivel de red para el Nodo Maestro MPI (activo-pasivo con failover automático).

Documento generado como base de un proyecto universitario de Sistemas Distribuidos — arquitectura SOA con servicios por capas, sin dependencias cloud ni contenedores.